

AI Agriculture Platform

Technical Specification / Architecture Blueprint

Version: 1.0 **Status:** Hackathon Foundation Architecture **Domain:** AI in Agriculture **Primary Goal:** Build a scalable, maintainable and extensible AI platform that can be adapted to different agricultural use cases.

1. Project Overview

The platform is designed as a reusable AI-powered agricultural system.

The exact business problem may vary depending on the hackathon challenge. The architecture must therefore support multiple use cases without major infrastructure changes.

Potential use cases include:

- Plant disease detection
- Crop recommendation
- Yield prediction
- Irrigation recommendations
- Soil analysis
- Weather-based agricultural recommendations
- Agricultural assistant / chatbot
- Image-based crop analysis
- Farm monitoring
- Agricultural document analysis
- IoT/sensor data processing

The system must support:

- Multiple concurrent users
- Horizontal scaling
- AI/LLM integration
- Semantic LLM caching
- RAG
- Image and document processing
- Asynchronous background jobs
- Load balancing
- Containerized deployment
- Kubernetes deployment
- Automatic scaling
- Monitoring and logging
- Secure API access
- Future integration with Kafka and IoT systems

2. Architectural Principles

The system follows these principles:

2.1 Modular Monolith

The main application is implemented as a modular monolith rather than a large set of microservices.

The application contains logically isolated modules:

Authentication
Users
Farms
Crops
Agricultural Analysis
AI
RAG
Semantic Cache
Notifications
File Management

Modules must have clear responsibilities and interfaces.

This allows the project to remain simple enough for a hackathon while preserving the possibility of extracting individual modules into independent services later.

2.2 Stateless Application

API instances must be stateless.

No important user/session/business state should be stored inside an API container.

Persistent state must be stored in:

- PostgreSQL
- Redis
- Object Storage
- Vector Storage

This allows multiple API replicas to run simultaneously.

2.3 Horizontal Scaling

The application must support:

API × N
Worker × N
AI Worker × N

Additional replicas can be started without changing application logic.

2.4 Asynchronous Processing

Long-running operations must not block HTTP requests.

Examples:

- Image analysis
- Large document processing
- Batch predictions
- Satellite image processing
- Large AI requests
- Report generation

These operations should be submitted to a queue.

2.5 AI Provider Independence

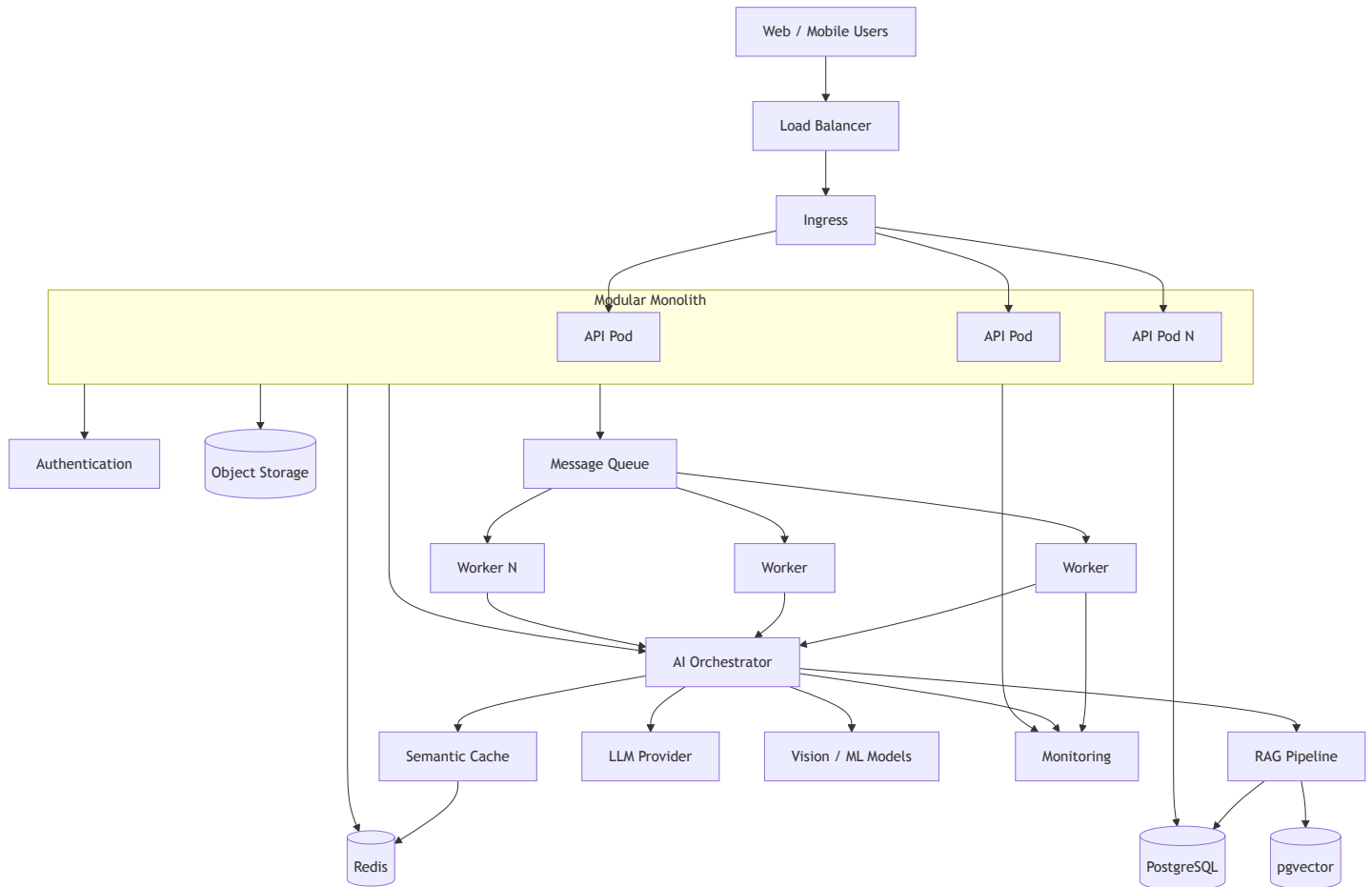
The application must not depend directly on a single LLM provider.

The AI layer should expose an abstraction:

```
AI Orchestrator
|
├── LLM Provider
├── Vision Model
├── Embedding Model
└── Local ML Model
```

This allows providers/models to be replaced without changing business logic.

3. High-Level Architecture



4. Technology Stack

4.1 Backend

Recommended:

Python
FastAPI
Pydantic
SQLAlchemy
Alembic

FastAPI is responsible for:

- REST API
- Request validation
- Authentication middleware
- API documentation
- Health checks
- Business logic orchestration

5. Database

PostgreSQL

PostgreSQL is the primary relational database.

It stores:

- Users
- Farms
- Fields
- Crops
- Agricultural observations
- AI analysis records
- Jobs
- Conversations
- Recommendations
- System configuration
- Audit information

5.1 Vector Search

Use:

PostgreSQL + pgvector

for the initial implementation.

This avoids introducing another database during the hackathon.

Vector storage can contain:

- Agricultural documents
- Crop information
- Disease information
- Research papers
- Knowledge base articles
- Previous AI requests
- Cached semantic representations

6. Redis

Redis is used for:

6.1 Regular Cache

Examples:

```
weather data
frequently accessed farm data
configuration
temporary results
```

6.2 Semantic Cache

Used for AI responses.

6.3 Rate Limiting

Example:

```
User:
100 requests / minute
```

6.4 Temporary State

Redis can be used for:

- Short-lived tokens
- Job status
- Distributed locks
- Temporary session data

7. Semantic LLM Cache

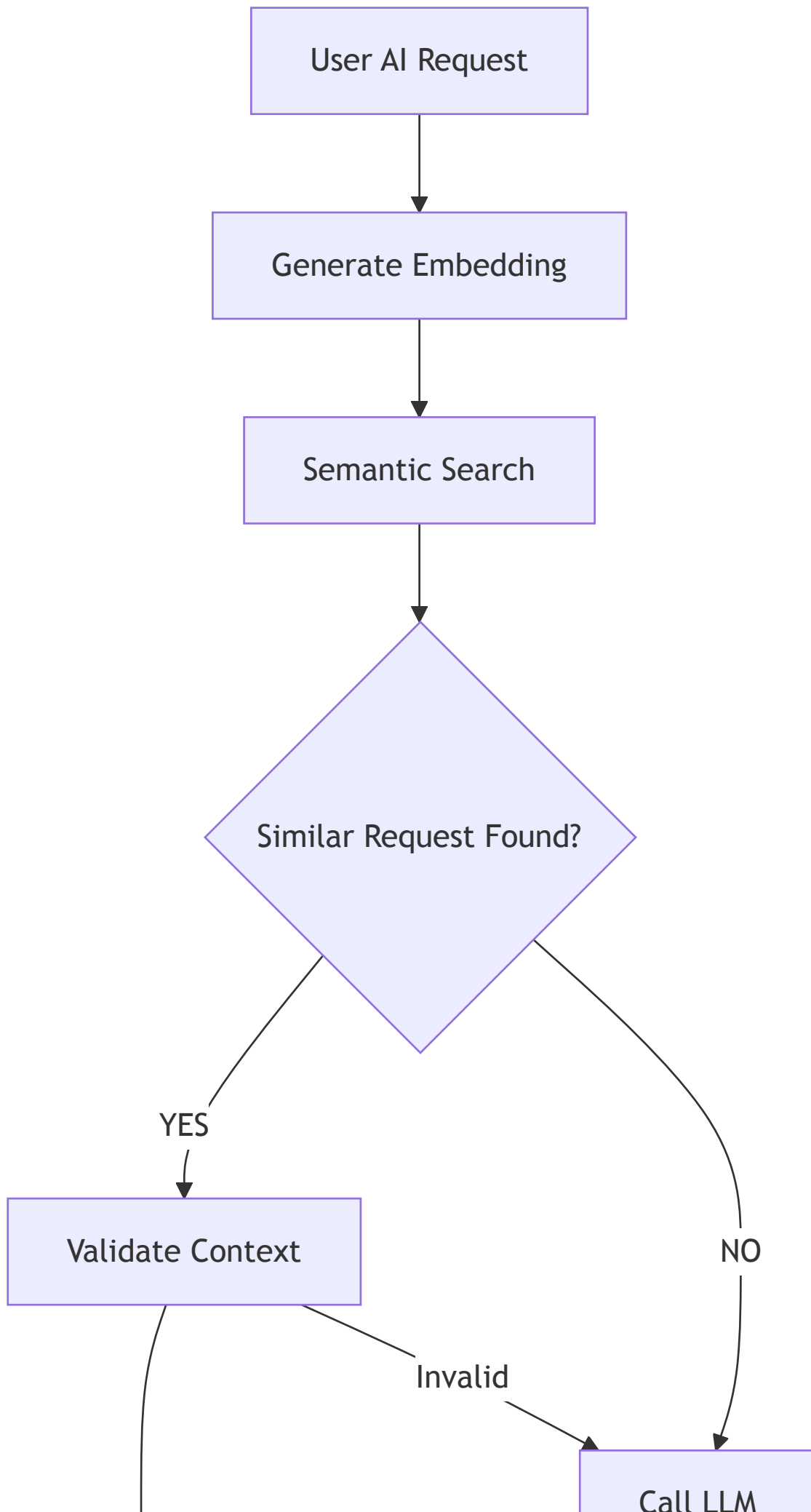
Objective

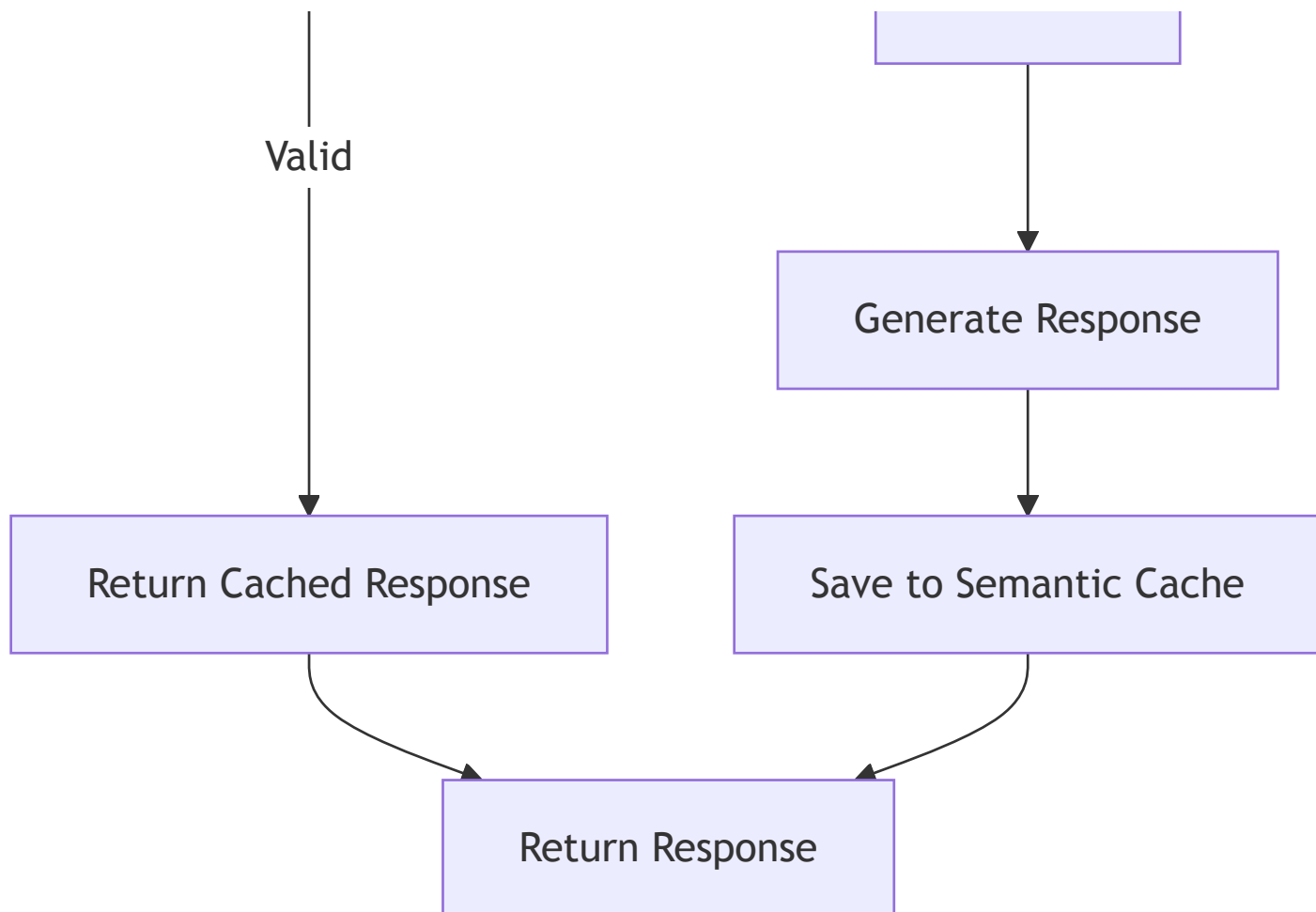
Reduce:

- LLM costs
- Response latency
- Number of external API requests

while preserving contextual correctness.

7.1 Basic Flow





8. Semantic Cache Context

The cache must not rely only on textual similarity.

Each cache record should contain metadata.

Example:

```
{
  "query": "Why are my wheat leaves turning yellow?",
  "embedding": "...",
  "response": "...",

  "crop": "wheat",
  "region": "Almaty",
  "country": "Kazakhstan",
  "language": "en",

  "season": "autumn",

  "model": "model-name",

  "created_at": "2026-09-21T10:00:00Z",
  "expires_at": "2026-09-28T10:00:00Z"
}
```

The cache must consider:

- Semantic similarity
- Crop
- Region
- Language
- Season
- Relevant environmental context
- Model version
- Cache expiration

9. Semantic Cache Decision

Example:

```
Similarity = 0.95  
Crop = wheat  
Region = Almaty  
Language = English  
Season = autumn
```

If all relevant context matches:

CACHE HIT

Otherwise:

CACHE MISS

The similarity threshold must be configurable.

Example:

```
SEMANTIC_CACHE_THRESHOLD=0.90  
SEMANTIC_CACHE_TTL=86400
```

10. AI Orchestrator

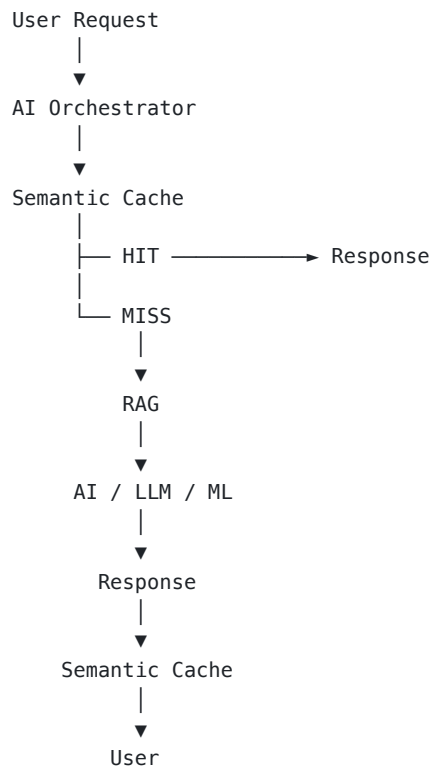
The AI Orchestrator is responsible for controlling the AI pipeline.

Responsibilities:

1. Analyze incoming request
2. Determine request type
3. Check semantic cache
4. Generate embeddings
5. Retrieve relevant knowledge
6. Select appropriate model

7. Execute AI inference
8. Validate result
9. Store result
10. Return response

10.1 AI Pipeline

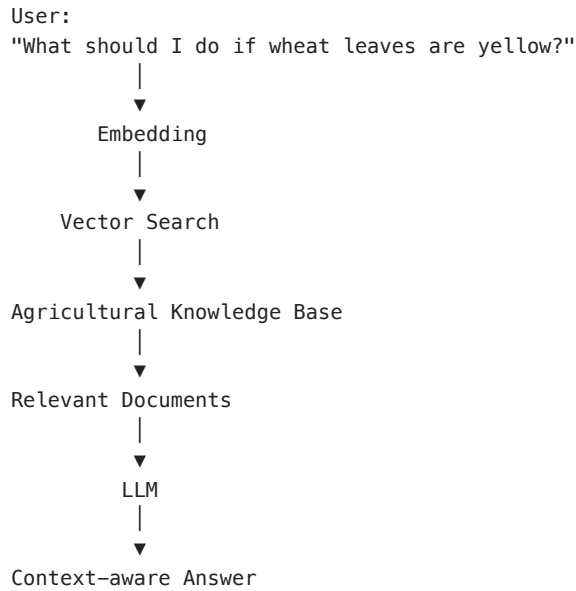


11. RAG

RAG = Retrieval-Augmented Generation.

The system retrieves relevant agricultural information before sending a request to the LLM.

Example:



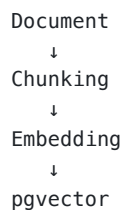
12. RAG Knowledge Base

The knowledge base can contain:

- Agricultural guidelines
- Crop information
- Disease information
- Regional recommendations
- Weather information
- Soil information
- Research papers
- Government agricultural documentation
- Internal expert knowledge

Each document should be split into chunks and converted into embeddings.

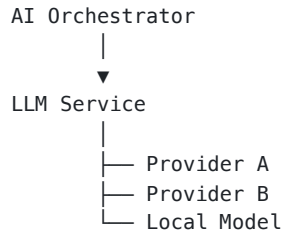
Example:



13. LLM Abstraction

Do not directly call an LLM from business logic.

Use:



Example interface:

```
class LLMProvider:

    async def generate(
        self,
        prompt: str,
        context: list[str],
        model: str
    ) -> str:
        ...
```

This allows the provider to be changed without modifying the rest of the application.

14. Image / Vision Processing

The platform must support agricultural images.

Examples:

- Plant photos
- Leaf photos
- Soil photos
- Field images
- Satellite images

Flow:

```
User
|
▼
Upload Image
|
▼
Object Storage
|
▼
Create Job
|
▼
Queue
|
▼
Vision Worker
|
▼
AI Model
|
▼
Analysis Result
|
▼
PostgreSQL
```

15. Object Storage

Object storage should be used for large files.

Examples:

```
/images
/documents
/satellite
/reports
/datasets
```

Recommended S3-compatible storage:

```
AWS S3
MinIO
Cloud Object Storage
```

Database should store metadata rather than large binary files.

Example:

```
{
  "file_id": "abc123",
  "storage_key": "farms/123/images/abc123.jpg",
  "content_type": "image/jpeg",
  "size": 1024000
}
```

16. Message Queue

The application requires asynchronous processing.

For the hackathon:

Redis Streams

or:

RabbitMQ

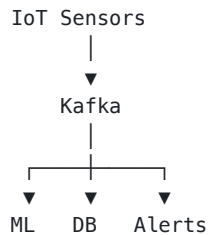
can be used.

Kafka is optional.

17. Kafka

Kafka should be introduced if the system needs high-volume event streaming.

Potential future use cases:



Examples:

- Soil moisture sensors
- Temperature sensors
- Humidity sensors
- Weather stations
- Farm telemetry
- GPS
- Satellite events

Kafka should not be introduced solely for architectural complexity.

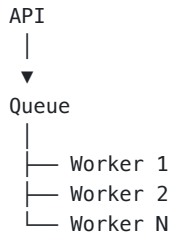
18. Background Workers

Workers execute long-running tasks.

Examples:

Image analysis
 Document processing
 Embedding generation
 Batch prediction
 Report generation
 Data ingestion
 AI inference

Architecture:



Workers must be horizontally scalable.

19. API Architecture

Use versioned REST APIs.

Base path:

`/api/v1`

Recommended endpoints:

POST	<code>/api/v1/auth/login</code>
POST	<code>/api/v1/auth/register</code>
GET	<code>/api/v1/users/me</code>
GET	<code>/api/v1/farms</code>
POST	<code>/api/v1/farms</code>
GET	<code>/api/v1/farms/{farm_id}</code>
GET	<code>/api/v1/crops</code>
POST	<code>/api/v1/crops</code>
POST	<code>/api/v1/analysis</code>
GET	<code>/api/v1/analysis/{analysis_id}</code>
POST	<code>/api/v1/chat</code>
GET	<code>/api/v1/chat/{conversation_id}</code>
POST	<code>/api/v1/files</code>
GET	<code>/api/v1/files/{file_id}</code>
GET	<code>/api/v1/jobs/{job_id}</code>
GET	<code>/health</code>
GET	<code>/ready</code>

20. Authentication

Recommended:

JWT

Authentication flow:



Every protected request must include:

Authorization: Bearer <token>

21. Authorization

Use role-based access control.

Example:

USER
FARM_OWNER
EXPERT
ADMIN

Permissions must be checked at the API/service layer.

Example:

Farm Owner

- └ View own farms
- └ Create analysis
- └ Upload images

Expert

- └ View assigned data
- └ Provide recommendations

Admin

- └ Manage users
- └ Manage knowledge base
- └ Monitor system

22. Error Handling

The API must return consistent error responses.

Example:

```
{
  "error": {
    "code": "INVALID_REQUEST",
    "message": "Invalid crop identifier",
    "request_id": "req-123"
  }
}
```

HTTP status codes:

```
200 OK
201 Created
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
422 Validation Error
429 Too Many Requests
500 Internal Server Error
503 Service Unavailable
```

23. Request ID

Every request should receive a unique ID.

Example:

```
X-Request-ID: 8f72b1...
```

The ID must appear in:

- Application logs
- Error logs
- Traces
- API responses where appropriate

This simplifies debugging.

24. Security

Minimum security requirements:

Authentication

JWT/OAuth2.

Passwords

Passwords must be hashed using a secure password hashing algorithm.

Secrets

Never store secrets in source code.

Use:

```
Kubernetes Secrets  
Environment Variables  
Secret Manager
```

API Security

Implement:

- Rate limiting
- Input validation
- File type validation
- File size limits
- Authentication
- Authorization
- CORS configuration
- Secure HTTP headers

25. Rate Limiting

Redis can be used for distributed rate limiting.

Example:

Anonymous:
20 requests/minute

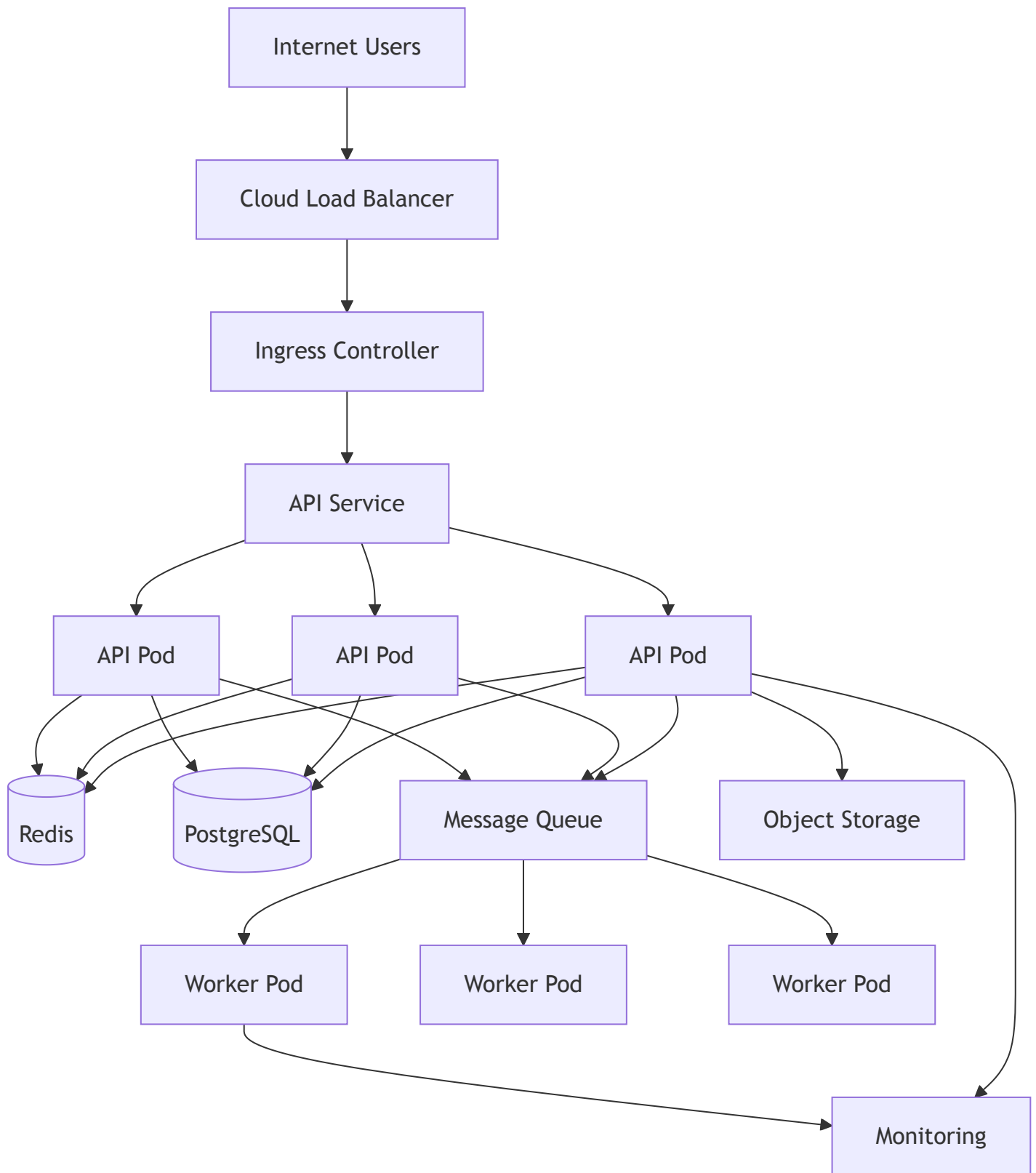
Authenticated:
100 requests/minute

AI:
20 requests/minute

Limits must be configurable.

26. Kubernetes Architecture

The production deployment must run in Kubernetes.



27. Kubernetes Components

Minimum components:

Deployment
Service
Ingress
ConfigMap
Secret
HorizontalPodAutoscaler
PersistentVolume
PersistentVolumeClaim

28. API Deployment

Example:

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: agriculture-api

spec:
  replicas: 3

  selector:
    matchLabels:
      app: agriculture-api

  template:
    metadata:
      labels:
        app: agriculture-api

    spec:
      containers:
        - name: api
          image: agriculture/api:latest

      ports:
        - containerPort: 8000

      resources:
        requests:
          cpu: "250m"
          memory: "256Mi"

        limits:
          cpu: "1000m"
          memory: "1Gi"
```

29. Kubernetes Service

The Service provides stable networking for API pods.

```
apiVersion: v1
kind: Service

metadata:
  name: agriculture-api

spec:
  selector:
    app: agriculture-api

  ports:
    - port: 80
      targetPort: 8000

type: ClusterIP
```

External traffic should normally reach the service through an Ingress/Load Balancer.

30. Horizontal Pod Autoscaler

API should scale automatically.

Example:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler

metadata:
  name: agriculture-api-hpa

spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: agriculture-api

  minReplicas: 2
  maxReplicas: 10

  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

Result:


```
Low traffic
  ↓
2 API pods

Medium traffic
  ↓
4 API pods

High traffic
  ↓
10 API pods
```

31. Worker Scaling

Workers should also scale independently.

Example:

```
API Pods:
2 → 10
```

```
Workers:
1 → 20
```

This is important because AI workloads may become the bottleneck rather than the API itself.

32. Health Checks

Every service must provide:

```
GET /health
GET /ready
```

Liveness

Determines whether the process is alive.

Readiness

Determines whether the service can receive traffic.

Example:

```
API
├─ PostgreSQL connection
├─ Redis connection
└─ Required dependencies
```

If readiness fails, Kubernetes stops sending traffic to that pod.

33. Rolling Deployment

Deployments must use rolling updates.

```
Old:
API1 API2 API3
```

Deploy new version:

```
API1(old) API2(old) API3(new)
```

```
API1(new) API2(old) API3(new)
```

```
API1(new) API2(new) API3(new)
```

This minimizes downtime.

34. Containerization

Every application component must have a Docker image.

Example:

```
backend/
Dockerfile
```

```
worker/
Dockerfile
```

The same image should be usable in:

```
Development
Staging
Production
```

Configuration must be provided through environment variables.

35. Configuration

No environment-specific values should be hardcoded.

Example:

```
DATABASE_URL=  
REDIS_URL=
```

```
LLM_API_KEY=  
LLM_MODEL=
```

```
S3_ENDPOINT=  
S3_BUCKET=  
S3_ACCESS_KEY=  
S3_SECRET_KEY=
```

```
SEMANTIC_CACHE_THRESHOLD=0.90  
SEMANTIC_CACHE_TTL=86400
```

```
MAX_FILE_SIZE_MB=20
```

36. Environment Separation

At minimum:

```
Development  
Staging  
Production
```

Example:

```
.env.dev  
.env.staging  
.env.prod
```

Secrets must not be committed to Git.

37. CI/CD

Use GitHub Actions or equivalent.

Pipeline:



38. CI Requirements

Every pull request should run:

- Unit Tests
- Integration Tests
- Linting
- Type Checking
- Docker Build
- Security Scan

Deployment should only happen if the required checks pass.

39. Monitoring

Use:

- Prometheus
- Grafana
- OpenTelemetry

Monitor:

Infrastructure

- CPU
- Memory
- Disk
- Network
- Pod count

API

- Request count
- Latency
- Error rate
- HTTP status codes

AI

- LLM requests
- LLM latency
- Token usage
- LLM errors
- Cache hit rate
- Cache miss rate
- Embedding latency
- RAG latency

Workers

Queue size
Job duration
Failed jobs
Worker utilization

40. Logging

Use structured JSON logs.

Example:

```
{
  "timestamp": "2026-09-21T10:30:00Z",
  "level": "INFO",
  "service": "ai-orchestrator",
  "request_id": "req-123",
  "user_id": "user-456",
  "event": "llm_request",
  "latency_ms": 820,
  "cache_hit": false
}
```

Do not log:

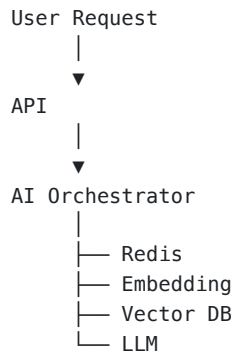
- Passwords
- API keys
- Tokens
- Sensitive personal information

41. Observability

The system should support:

Logs
+
Metrics
+
Traces

Example:



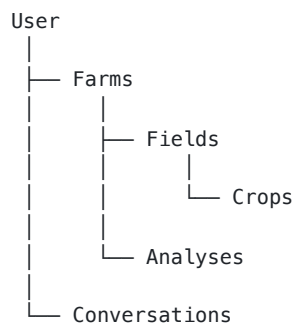
Tracing should allow developers to identify which component caused latency.

42. Database Design

Initial entities:

User
Farm
Field
Crop
CropObservation
Analysis
AIRequest
AIResponse
Conversation
Message
File
Job
KnowledgeDocument
KnowledgeChunk

Example relationships:



43. AI Analysis Entity

Example:

```
{
  "id": "analysis-123",
  "farm_id": "farm-123",
  "crop": "wheat",
  "type": "disease_detection",
  "status": "completed",
  "result": {},
  "model": "vision-model-v1",
  "created_at": "..."
}
```

44. Job System

Long-running tasks must have a job lifecycle.

```
PENDING
  ↓
PROCESSING
  ↓
COMPLETED
```

or:

```
PROCESSING
  ↓
FAILED
```

Optional:

```
CANCELLED
```

Example:

```
{
  "job_id": "job-123",
  "status": "processing",
  "progress": 65
}
```

45. AI Safety / Reliability

The AI layer should not blindly return generated information.

The system should:

- Provide relevant context through RAG
- Include source information where possible
- Distinguish AI-generated recommendations from verified data

- Validate structured AI outputs
- Handle model failures
- Set request timeouts
- Retry transient failures
- Use fallback models where appropriate

For agricultural recommendations that could materially affect crops or finances, the UI should clearly indicate that AI output is advisory and may require expert verification.

46. AI Response Structure

Prefer structured output.

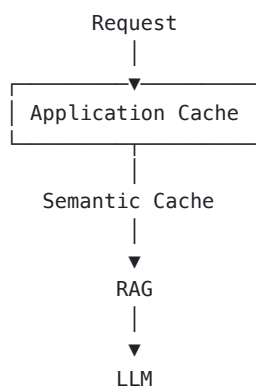
Example:

```
{
  "summary": "...",
  "possible_causes": [
    "...",
    "..."
  ],
  "recommendations": [
    "...",
    "..."
  ],
  "confidence": 0.87,
  "sources": [
    {
      "title": "...",
      "reference": "..."
    }
  ]
}
```

The exact schema depends on the hackathon use case.

47. Caching Strategy

Use several cache levels.



Cache types

Exact Cache
 Semantic Cache
 Database Cache
 External API Cache

TTL must depend on data type.

For example:

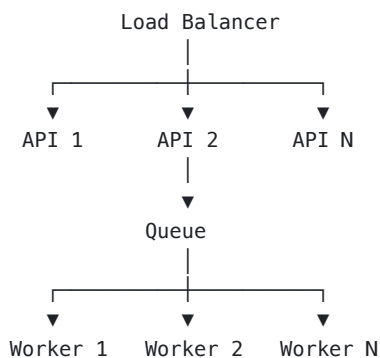
Static agricultural information:
 long TTL

Weather:
 short TTL

AI recommendations:
 context-dependent TTL

48. Scalability Strategy

The platform scales independently at different layers.



Database scaling can later include:

Read replicas
 Connection pooling
 Partitioning
 Database optimization

Redis can be deployed in a highly available configuration when required.

49. Database Connection Pooling

API instances must use connection pooling.

Without pooling:

100 API requests
→ 100 database connections

With pooling:

100 API requests
↓
Connection Pool
↓
20 DB connections

Pool parameters must be configurable.

50. Disaster Recovery

Production deployment should include:

Database

Automated backups
Point-in-time recovery where supported

Object Storage

Versioning
Replication / backup

Kubernetes

Application configuration should be stored as code:

Helm
Kubernetes manifests
Git

This allows infrastructure to be recreated.

51. Infrastructure as Code

Recommended:

Terraform

for cloud infrastructure.

Potential resources:

Kubernetes Cluster
Load Balancer
Database
Redis
Object Storage
Container Registry
Networking
Monitoring

For the hackathon, Terraform can be presented as the production deployment layer even if the MVP uses a simpler environment.

52. Recommended Repository Structure

```
agriculture-ai/
├── backend/
│   ├── app/
│   │   ├── api/
│   │   ├── auth/
│   │   ├── farms/
│   │   ├── crops/
│   │   ├── analysis/
│   │   ├── ai/
│   │   ├── rag/
│   │   ├── cache/
│   │   ├── storage/
│   │   ├── workers/
│   │   ├── database/
│   │   └── main.py
│   ├── tests/
│   ├── Dockerfile
│   └── pyproject.toml
├── frontend/
├── infrastructure/
│   ├── docker/
│   ├── kubernetes/
│   │   ├── namespace.yaml
│   │   ├── configmap.yaml
│   │   ├── secret.yaml
│   │   ├── api-deployment.yaml
│   │   ├── api-service.yaml
│   │   ├── worker-deployment.yaml
│   │   ├── ingress.yaml
│   │   └── hpa.yaml
│   └── terraform/
├── docs/
│   ├── architecture.md
│   ├── api.md
│   └── ai.md
├── docker-compose.yml
└── README.md
```

53. Development Environment

Local environment:

Docker Compose

```
|— backend
|— worker
|— postgres
|— redis
|— minio
```

Run:

```
docker compose up
```

The local environment should reproduce the main production dependencies.

54. Production Environment

```
Cloud
|
|— Kubernetes
|   |— API Pods
|   |— Worker Pods
|   |— AI Pods if required
|
|— Managed PostgreSQL
|
|— Managed Redis
|
|— Object Storage
|
|— Load Balancer
|
|— Monitoring
```

55. MVP Scope

For the hackathon, the minimum implementation should contain:

Backend

- FastAPI
- PostgreSQL
- Redis
- JWT authentication
- REST API

AI

- AI Orchestrator
- LLM integration
- Embeddings
- Semantic cache
- Basic RAG

Files

- Object storage
- Image upload

Async

- Queue
- Worker

Infrastructure

- Docker
- Kubernetes manifests
- Load Balancer / Ingress
- HPA

Observability

- Health checks
- Logging
- Basic metrics

56. Optional Components

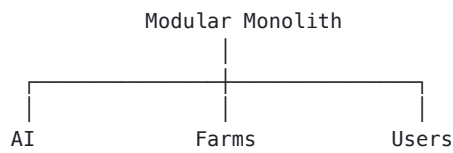
These should only be implemented if time permits:

Kafka
Terraform
Advanced monitoring
Multiple LLM providers
Advanced ML models
IoT integration
Satellite data
Read replicas
Advanced autoscaling
Multi-region deployment

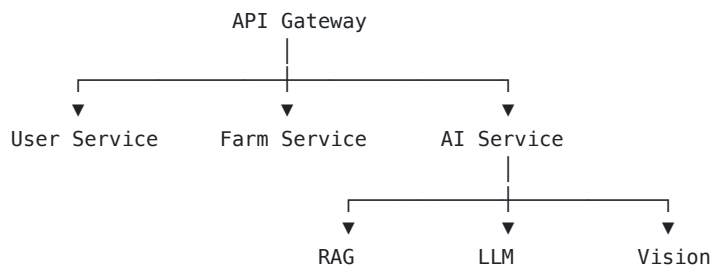
57. Future Architecture

When the platform grows, modules can be extracted.

Initial:



Future:



The extraction should happen only when there is a real need for independent scaling, deployment or ownership.

58. Architecture Decision Summary

Decision	Initial Solution	Future
Application	Modular Monolith	Microservices where justified
API	FastAPI	Same
Database	PostgreSQL	Read replicas / scaling
Vector DB	pgvector	Dedicated Vector DB if required
Cache	Redis	Redis Cluster
Queue	Redis/RabbitMQ	Kafka for event streaming
AI	AI Orchestrator	Dedicated AI Service
Files	S3/MinIO	Cloud Object Storage
Deployment	Docker	Kubernetes
Scaling	Kubernetes HPA	Advanced autoscaling
Monitoring	Prometheus/Grafana	Full observability stack
IaC	Optional	Terraform
CI/CD	GitHub Actions	GitOps if required

59. Key Non-Functional Requirements

Performance

Target:

API p95 latency < 500 ms

for regular non-AI requests under normal load.

AI latency depends on the selected model and external provider.

Availability

Application should avoid single points of failure at the application layer.

Minimum:

API replicas >= 2

for production.

Scalability

System must support horizontal scaling of:

API
Workers
AI Workers

without application redesign.

Maintainability

Requirements:

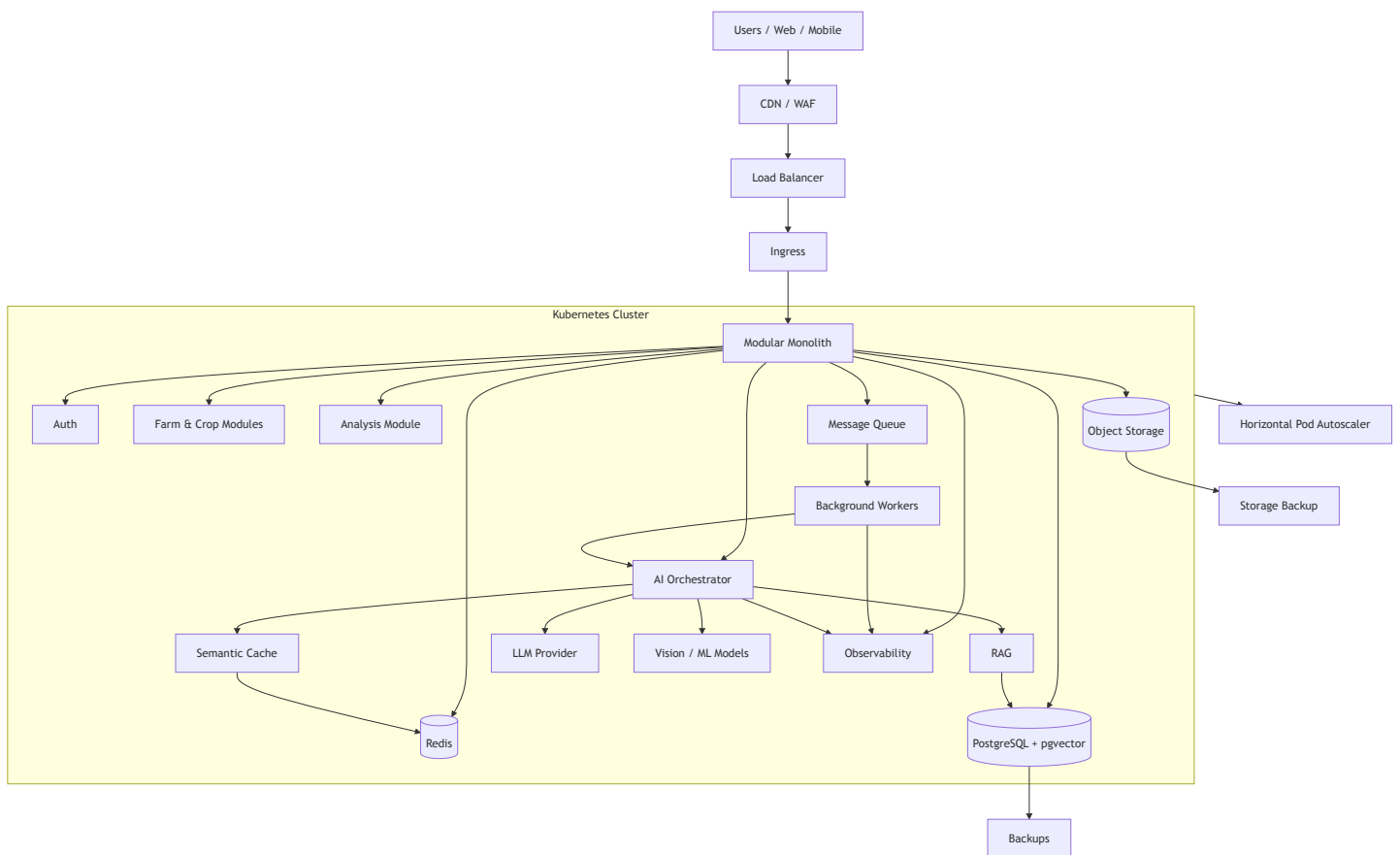
- Modular code
- Clear service boundaries
- Automated tests
- API versioning
- Configuration through environment variables
- Documentation
- Structured logging

Security

Requirements:

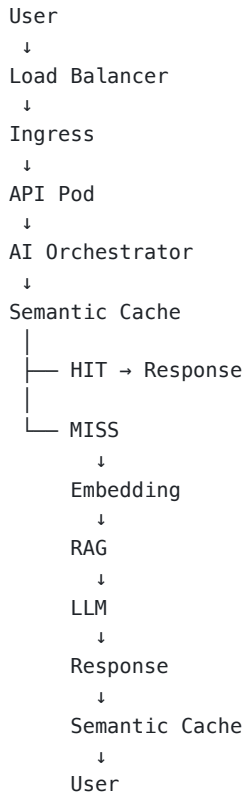
- Authentication
- Authorization
- Rate limiting
- Input validation
- Secure secret management
- HTTPS
- No secrets in Git
- File validation

60. Final Architecture

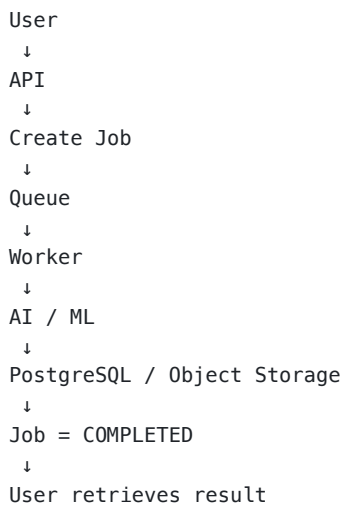


61. Target Request Flow

For a normal AI request:



For a heavy task:



62. Recommended Hackathon Implementation Order

Phase 1 — Core

FastAPI
 PostgreSQL
 Docker
 Basic API

Phase 2 — AI

LLM
AI Orchestrator
Embeddings
RAG

Phase 3 — Performance

Redis
Semantic Cache
Rate Limiting

Phase 4 — Async

Queue
Workers
Background processing

Phase 5 — Deployment

Docker
Kubernetes
Ingress
HPA

Phase 6 — Observability

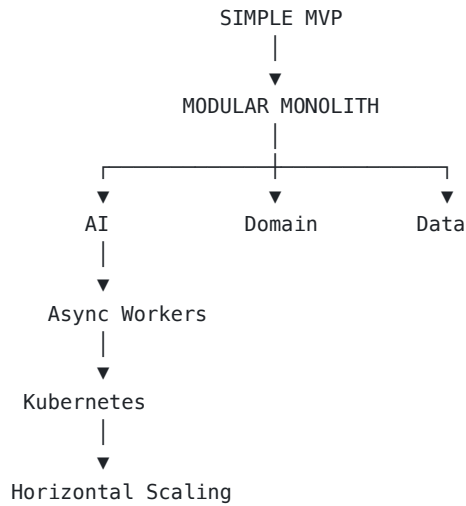
Logging
Prometheus
Grafana
Health checks

Phase 7 — Optional

Kafka
IoT
Advanced ML
Terraform
Multiple AI providers

63. Core Architecture Principle

The platform should follow:



The system should **not start as a distributed microservice architecture**.

The initial goal is:

Simple enough to build during a hackathon, but structured enough to evolve into a production-scale platform.

The exact agricultural functionality can then be plugged into the AI , Analysis , RAG , and domain modules once the final hackathon challenge is announced.

Если тема хакатона окажется, например, **plant disease detection, smart irrigation, yield prediction** или **AI farmer assistant**, менять придется в основном только Analysis/AI часть — базовые Kubernetes, cache, queue, database, CI/CD и scaling можно оставить такими же.